

3주차 과제-2 (c)

사람 김은아(엘텍공과대학 소프트웨어학부)

Manifest 파싱



Androguard 4.1.4 설치 하시오!

scale 정할 필요 有

*깃허브에 `static_analyzer/` 패키지 만들어지면 그 안에 `manifest_parser.py` 만들어서 지금 만든 딕셔너리+함수 전부 넣을 예정.

1. 라이브러리 선택 (androguard vs. ElementTree)

▼ androguard 설치 및 파싱 코드

- 설치 (약 1분 내외)

```
pip install androguard # 4.1.4 버전 설치됐음
```

- 실제 APK 파일을 열어서 그 안의 정보(권한, 컴포넌트, Manifest 내용 등)를 꺼낼 수 있는 객체로 만들기: 실제 실행(테스트)할 때 사용함

```
from androguard.core.apk import APK

# 디컴파일 없이 APK 파일 자체를 바로 읽음
# 나중에 실제로 분석 진행할 APK 파일 이름을 ""안에 넣으면 됨
apk = APK("sample.apk")
```

▼ androguard 선택한 이유

권한/컴포넌트 파싱 API가 이미 있어서 직접 XML 태그 뒤지는 것보다 훨씬 빠르고 안전함. 바이너리 AXML도 알아서 처리해 줌.

ElementTree는 바이너리 AXML을 못 읽어서 apktool로 디컴파일된 텍스트 XML이 필요함.

2. 위험 권한 목록 + 가중치 테이블 정의

- 딕셔너리 생성 (`PERMISSION_WEIGHTS` - C 내부에서만 쓰는 참고표) - 3번 코드에 함께 넣어둠.

*숫자 스케일은 D와 맞춰야 함. (1~10인지 1~5인지)

3. 권한 추출 로직, 컴포넌트 + 인텐트 필터 + exported 체크

: `parse_manifest` 안으로 합침.

* `exported="true"` 인데 `intent-filter` 도 있는 컴포넌트가 특히 위험함. (외부에서 임의로 호출 가능)

→ D가 가중치에 반영할 수 있음

▼ manifest_parser.py

```
from androguard.core.apk import APK

PERMISSION_WEIGHTS = {
    "android.permission.BIND_ACCESSIBILITY_SERVICE": 10,
    "android.permission.READ_SMS": 9,
    "android.permission.RECEIVE_SMS": 9,
    "android.permission.SYSTEM_ALERT_WINDOW": 10,
    "android.permission.REQUEST_INSTALL_PACKAGES": 5,
    # 필요한 만큼 계속 추가하면 됨
}

NS = "{http://schemas.android.com/apk/res/android}"

def _get_component_info(manifest_root, tag_name):
    """이름 목록 + exported 여부를 함께 뽑는 내부 헬퍼"""
    result = []
    for elem in manifest_root.iter(tag_name):
        name = elem.get(f"{NS}name")
        exported_attr = elem.get(f"{NS}exported")
        has_intent_filter = elem.find("intent-filter") is not None

        if exported_attr is not None:
            exported = exported_attr == "true"
        else:
            exported = has_intent_filter

        result.append({"name": name, "exported": exported})
    return result

def parse_manifest(apk_path: str) -> dict:
    apk = APK(apk_path)
    manifest_root = apk.get_android_manifest_xml()

    # 권한
    all_perms = apk.get_permissions()
    dangerous_perms = [p for p in all_perms if PERMISSION_WEIGHTS.get(p, 0) >= 8]

    # 컴포넌트별 정보 (내부적으로만 exported 판단에 사용)
    activities_info = _get_component_info(manifest_root, "activity")
    services_info = _get_component_info(manifest_root, "service")
    receivers_info = _get_component_info(manifest_root, "receiver")
    providers_info = _get_component_info(manifest_root, "provider")

    # exported인 것만 이름만 모아서 통합 리스트로
    exported_components = [
        c["name"]
        for group in (activities_info, services_info, receivers_info, providers_info)
        for c in group
        if c["exported"]
    ]
```

```

return {
    "permissions": all_perms,
    "dangerous_permissions": dangerous_perms,
    "activities": [c["name"] for c in activities_info],
    "services": [c["name"] for c in services_info],
    "receivers": [c["name"] for c in receivers_info],
    "providers": [c["name"] for c in providers_info],
    "exported_components": exported_components,
}

```

4. C의 출력 형식 (A 기반)

- 필드명 그대로, 이 함수를 실행한 결과값(딕셔너리)을 D가 받아서 `risk_score` 계산 후 최종 `static_report.json`에 합치는 구조. (`parse_manifest`)
- 리턴 딕셔너리 형식을 `schema.py`에 타입으로 정의해서 팀 전체가 import해서 쓰는 구조로 가도록.
- 출력은 총 7개.

```

parse_manifest(apk_path) → dict
|
├─ permissions: list[str]           # 앱이 요청하는 모든 권한 문자열
├─ dangerous_permissions: list[str]  # permissions 중 가중치 8점 이상인 것만 필터링
├─ activities: list[str]            # 액티비티 이름만
├─ services: list[str]              # 서비스 이름만
├─ receivers: list[str]             # 리시버 이름만
├─ providers: list[str]             # Content Provider 이름만
└─ exported_components: list[str]    # activities/services/receivers/providers 중
                                     # exported=true인 것들만 이름만 모아 통합

```

→ 권한이 하나도 없는 앱이면 `all: []`, 컴포넌트가 없으면 `activities: []` 등 빈 리스트로 나올 것. (키 자체가 사라지지는 않는다.)

* `exceptions.py` 안에 이미 `StaticAnalysisError`가 정의되어 있는지 A한테 확인하기. (어떤 예외 클래스를 쓸지 맞춰야 함)

- `manifestInfo` TypedDict랑 키 이름, 타입 다 일치하는지 한번 더 대조가 필요함.

6. 검증 (추후에)

- 정상 앱 2~3개, 공개 악성 샘플 2~3개로 돌려보고, 고위험 권한 개수/exported 컴포넌트 개수에 실제로 차이가 나는지 확인한다.
- 여기에서 차이가 안 보이면 가중치 테이블부터 다시 봐야 함.

▼ 샘플 APK 구하는 경로

1. 정상 사이트에서 무료 안드로이드 앱 다운로드. (APKPure, APIMirror)
2. 에뮬레이터에 이미 깔려있는 시스템 앱을 뽑아도 됨(adb, emulator 실행 중일 때) :

```
adb shell pm list packages
adb shell pm path <com.android.chrome> # 리스트에서 찾은 앱 중 하나 넣으면 됨
adb pull /data/app/.../base.apk sample.apk # 방금 찾은 경로 복사
```

예시)

```
package:com.google.android.apps.restore
PS C:\Users\User> adb shell pm path com.google.android.apps.restore
package:/product/priv-app/GoogleRestorePrebuilt/GoogleRestorePrebuilt.apk
PS C:\Users\User> adb pull /product/priv-app/GoogleRestorePrebuilt/GoogleRestorePrebuilt.apk
/product/priv-app/GoogleRestore... MB/s (5108286 bytes in 0.331s)
```

▼ 에뮬레이터 백그라운드에서 실행 방법

```
adb version
emulator -list-avds # 에뮬레이터 목록

# 에뮬레이터 띄우는 법 2가지
emulator -avd 에뮬레이터이름 # 창 0
emulator -avd 에뮬레이터이름 -no-window
# 창 x

# 다른 cmd/터미널 창에서 켜졌는지 확인
adb devices

# 어떤 AVD가 5554에서 돌고 있는지 확인
adb -s emulator-5554 emu avd name
# 포트 번호는 본인 실행 환경 생각해서 변경하기

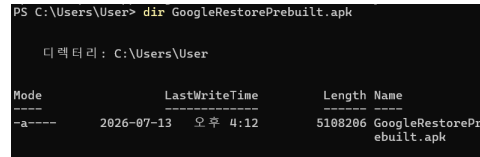
# 사용자가 설치한 앱만 보여줌
adb shell pm list packages -3
```

함수 검증 과정 (다시 필요함 7/13 21:28 기준)

▼ 정상 샘플로 확인한 결과

1. 파일 위치 확인

```
dir GoogleRestorePrebuilt.apk
```



```
PS C:\Users\User> dir GoogleRestorePrebuilt.apk

디렉터리: C:\Users\User

Mode                LastWriteTime         Length Name
----                -
-a-----         2026-07-13 오후 4:12      5188286 GoogleRestorePrebuilt.apk
```

2. 테스트용 폴더 구조 잡기

- 샘플 파일은 user\samples\
- user\static_analyzer\manifest_parser.py

```
mkdir samples
move GoogleRestorePrebuilt.apk samples\

mkdir static_analyzer
New-Item static_analyzer\__init__.py # 빈 파일
New-Item static_analyzer\manifest_parser.py
# 파일 열어서 코드 수정
code static_analyzer\manifest_parser.py

New-Item test_manifest.py
code test_manifest.py
```

3. 결과 (성공)

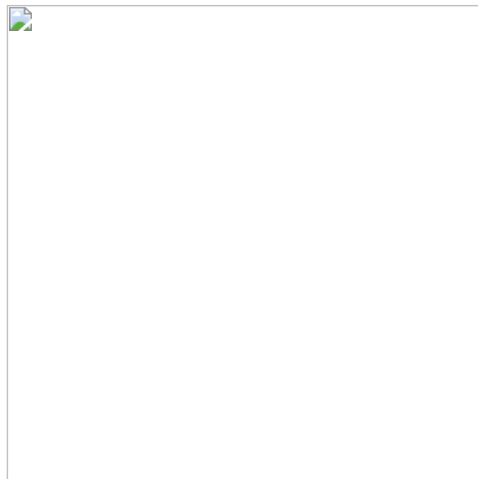
- permissions: 정상적으로 19개 권한 다 뽑힘. `high_risk` 는 `[]` (빈 배열)로 나왔는데, 정상 샘플이므로 고위험 권한이 없는 게 당연함.

```
=== permissions ===
{'all': ['android.permission.GET_ACCOUNTS_PRIVILEGED', 'com.google.android.setupwizard.READ_DEVICE_ORIGIN_FIRST_PARTY', 'android.permission.READ_CONTACTS', 'android.permission.READ_PRIVILEGED_PHONE_STATE', 'android.permission.WAKE_LOCK', 'com.google.android.setupwizard.SETUP_COMPAT_SERVICE', 'android.permission.WRITE_CONTACTS', 'android.permission.INTERNET', 'com.google.android.providers.gsf.permission.READ_GSERVICES', 'android.permission.ACCESS_NETWORK_STATE', 'android.permission.WRITE_EXTERNAL_STORAGE', 'android.permission.BACKUP', 'android.permission.WRITE_SECURE_SETTINGS', 'android.permission.WRITE_EMBEDDED_SUBSCRIPTIONS', 'com.google.android.setupwizard.WRITE_DEVICE_ORIGIN', 'android.permission.READ_EXTERNAL_STORAGE', 'android.permission.RECEIVE_BOOT_COMPLETED', 'com.android.vending.setup.PLAY_SETUP_SERVICE', 'android.permission.SUBSTITUTE_NOTIFICATION_APP_NAME'], 'high_risk': []}]

=== components ===
{'activities': [{'name': 'com.google.android.apps.pixelmigrate.component.FlowChoiceActivity', 'exported': True, 'has_intent_filter': True}, {'name': 'com.google.android.apps.pixelmigrate.common.component.WorkProfileSetupActivity', 'exported': True, 'has_intent_filter': True}, {'name': 'com.google.android.apps.pixelmigrate.component.RestoreChoiceActivity', 'exported': True, 'has_intent_filter': True}, {'name': 'com.google.android.apps.pixelmigrate.cloudrestore.component.CloudRestoreFlowActivity', 'exported': True, 'has_intent_filter': True}, {'name': 'com.google.android.apps.pixelmigrate.cloudrestore.component.KeyRecoveryLockScreenEntryActivity', 'exported': True, 'has_intent_filter': True}]}
```

```
d': True, 'has_intent_filter': False}, {'name': 'com.google.android.apps.pixelmigrate.component.StubLauncherActivity', 'exported': True, 'has_intent_filter': True}, {'name': 'com.google.android.apps.pixelmigrate.cloudrestore.component.AddAccountActivity', 'exported': True, 'has_intent_filter': True}, {'name': 'com.google.android.apps.pixelmigrate.cloudrestore.component.AppPickerHostActivity', 'exported': True, 'has_intent_filter': True}, {'name': 'com.google.android.gms.common.api.GoogleApiActivity', 'exported': False, 'has_intent_filter': False}, {'name': 'com.google.android.libraries.performance.primes.debug.PrimesEventActivity', 'exported': False, 'has_intent_filter': True}], 'services': [{'name': 'com.google.android.apps.pixelmigrate.cloudrestore.component.CloudRestoreService', 'exported': False, 'has_intent_filter': False}, {'name': 'com.google.firebase.components.ComponentDiscoveryService', 'exported': False, 'has_intent_filter': False}], 'receivers': [{'name': 'com.google.android.apps.pixelmigrate.component.ResetFlowReceiver', 'exported': True, 'has_intent_filter': True}, {'name': 'com.google.android.apps.pixelmigrate.component.LauncherRestoreStartReceiver', 'exported': True, 'has_intent_filter': True}, {'name': 'com.google.android.apps.pixelmigrate.component.LauncherRestoreFinishReceiver', 'exported': True, 'has_intent_filter': True}, {'name': 'com.google.android.apps.pixelmigrate.component.PhenotypeUpdateReceiver', 'exported': True, 'has_intent_filter': True}, {'name': 'com.google.android.apps.pixelmigrate.cloudrestore.googleone.GoogleOneNotificationClickReceiver', 'exported': False, 'has_intent_filter': False}, {'name': 'com.google.firebase.iid.FirebaseInstanceIdReceiver', 'exported': True, 'has_intent_filter': True}]}
```

- components: activities(10), services(2), receivers(6) 전부 뽑힘. `exported` / `has_intent_filter` 값도 각각 T/F 섞여서 나왔으므로 로직이 제대로 구분하고 있음을 확인.
 - `PrimesEventActivity` — `exported=False`인데 `intent-filter=True` (*인텐트 필터 있어도 명시적으로 `exported="false"` 박아놓은 경우니까, 아까 짰 "명시 안 했을 때만 인텐트필터로 기본값 판단" 로직이 제대로 작동했다는 증거가 됨)



▼ 악성 샘플(anubis.apk)로 확인한 결과 (아직 x)